

Configuration Management: Spring Cloud Config

Understand the actual problem first:

How do you manage configuration for multiple services and multiple environments without hardcoding it inside each application?

1. What is Configuration Management?

A Spring Boot application has configuration such as:

```
server:
  port: 8080

database:
  url: jdbc:mysql://...

payment:
  timeout: 5000
```

The problem starts when you have multiple environments:

```
Development
Testing
UAT
Production
```

Each environment may have different:

```
DB URL
API URL
credentials/secrets
timeout
feature flags
logging levels
```

You don't want:

```
application-dev.yml
application-test.yml
application-uat.yml
application-prod.yml
```

inside every microservice with duplicated configuration and difficult maintenance.

2. What is Spring Cloud Config?

Spring Cloud Config provides a centralized way to manage **external configuration for distributed applications**.

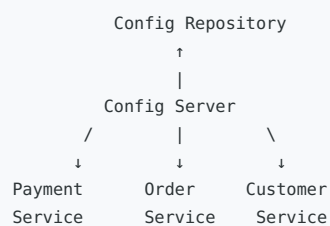
Instead of:

```
Payment Service
└─ config

Order Service
└─ config

Customer Service
└─ config
```

you can have:



The services retrieve their configuration from the Config Server.

3. Architecture

The main components are:

```
Config Repository
  ↓
Config Server
  ↓
Microservice
```

The repository can commonly be Git-backed:

```
Git Repository
  ↓
Spring Cloud Config Server
  ↓
Spring Boot Client
```

4. Why Do We Need a Config Server?

Suppose you have 20 microservices.

Without centralized configuration:

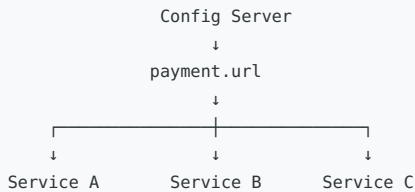
```
Service 1 → config
```

```
Service 2 → config
Service 3 → config
...
Service 20 → config
```

Now suppose the production payment-provider URL changes.

You have to find every application containing that configuration.

With Config Server:



Configuration can be managed centrally.

5. Typical Spring Cloud Config Architecture

Example repository:

```
config-repo/
|
├─ application.yml
├─ payment-service.yml
├─ payment-service-dev.yml
├─ payment-service-prod.yml
└─ order-service.yml
```

The Config Server reads this repository.

Then:

```
Payment Service
  ↓
Config Server
  ↓
payment-service-prod.yml
```

6. Server vs Client

This distinction is important.

Config Server

Responsible for:

- retrieving configuration
- exposing configuration to clients
- loading it from a backend such as Git

Config Client

Your actual Spring Boot application:

```
Payment Service  
Order Service  
BPS
```

They act as clients of Config Server.

7. How Configuration Is Selected

Spring Cloud Config commonly uses:

```
application name  
+  
profile  
+  
label
```

Conceptually:

```
payment-service  
+  
prod  
+  
main
```

determines which configuration is returned.

For example:

```
payment-service-prod.yml
```

8. Generic Example

Suppose the Config repository contains:

```
payment:  
  gateway-url: https://prod-payment.example.com  
  timeout: 5000
```

Your Spring Boot service can read:

```
@Value("${payment.gateway-url}")  
private String paymentGatewayUrl;
```

or preferably for structured configuration:

```
@ConfigurationProperties(prefix = "payment")  
public class PaymentProperties {  
  
    private String gatewayUrl;
```

```
private Duration timeout;

// getters/setters
}
```

9. @Value vs @ConfigurationProperties

This is a useful Spring interview topic.

@Value

Good for individual properties:

```
@Value("${payment.timeout}")
private int timeout;
```

@ConfigurationProperties

Better when you have a group of related properties:

```
payment:
  gateway-url: ...
  timeout: 5s
  retry-count: 3
```

```
@ConfigurationProperties(prefix = "payment")
public class PaymentProperties {

    private String gatewayUrl;
    private Duration timeout;
    private int retryCount;
}
```

Benefits:

- strongly structured configuration
- type conversion
- better maintainability
- easier validation

10. Environment-Specific Configuration

Suppose:

DEV

```
payment:
  gateway-url: http://localhost:8081
  timeout: 2s
```

PROD

```
payment:
  gateway-url: https://payment-prod.example.com
  timeout: 5s
```

The application doesn't need different Java code.

It loads the configuration appropriate for:

```
spring.profiles.active=prod
```

Conceptually:

```
Same code
↓
different profile
↓
different configuration
```

11. Configuration vs Secrets

This distinction is **very important**.

Not every configuration value should be stored as plain text in Git.

For example:

```
Service URL
Timeout
Feature flag
```

are configuration.

But:

```
Database password
API secret
Private key
OAuth client secret
```

are **secrets**.

Don't put:

```
db:
  password: MyProductionPassword123
```

into a normal Git repository.

Instead, production systems commonly use:

- Vault
- AWS Secrets Manager
- Azure Key Vault
- Kubernetes Secrets
- other secret-management solutions

Spring Cloud Config can participate in configuration management, but it shouldn't be treated as a

universal replacement for a secrets manager.

12. Security of Config Server

The Config Server itself contains potentially sensitive information.

Therefore protect it with:

```
Authentication
Authorization
TLS
Network controls
```

You don't want:

```
Internet
↓
Config Server
↓
Database credentials
```

publicly accessible.

13. Refreshing Configuration

A common question:

"If configuration changes, does the application need to restart?"

There are mechanisms for refreshing configuration in some Spring Cloud setups.

Historically:

```
/actuator/refresh
```

could be used with refresh-scoped configuration.

Spring Cloud also provides patterns involving:

```
@RefreshScope
```

and Spring Cloud Bus can help propagate refresh events across multiple applications.

Conceptually:

```
Config changed
↓
Config Server
↓
Refresh event
↓
Services
↓
Reload configuration
```

But not every property is safely refreshable. Some changes require restarting the application, especially configuration that affects initialization or underlying infrastructure.

14. Spring Cloud Bus

Suppose you have:

```
20 services
```

and configuration changes.

You don't want to manually refresh:

```
Service 1  
Service 2  
...  
Service 20
```

Spring Cloud Bus can propagate configuration change events through a messaging infrastructure.

Conceptually:

```
Config change  
  ↓  
Bus  
  ↓  
Service A  
Service B  
Service C  
...
```

This is an advanced topic; understand the purpose rather than memorizing implementation details.

15. Configuration Priority

Spring Boot supports multiple property sources.

For example:

```
application.yml  
environment variables  
command-line arguments  
external configuration
```

The exact precedence depends on the source and Spring Boot version.

The important concept is:

Spring resolves configuration from multiple property sources according to its precedence rules.

Don't casually say:

"application.yml always wins."

It doesn't.

16. Config Server vs Environment Variables

This is an excellent interview question.

Environment variables

Good for:

```
deployment-specific settings
secrets injection
containerized applications
```

Example:

```
PAYMENT_TIMEOUT=5000
```

Config Server

Good for:

```
centralized shared configuration
versioned configuration
environment-specific configuration
configuration shared across services
```

They can coexist.

For example:

```
Git/Config Server
→ normal application configuration

Secret Manager
→ secrets

Environment variables
→ deployment overrides
```

17. Config Server vs Service Discovery

Don't confuse them.

Service Discovery

Answers:

Where is a service?

Example:

```
payment-service
  ↓
instances
```

Config Server

Answers:

What configuration should this service use?

Example:

```
payment.timeout=5s
payment.url=...
```

Remember:

```
Service Discovery
→ LOCATION

Config Server
→ CONFIGURATION
```

18. Config Server vs API Gateway

Again, different responsibilities.

```
API Gateway
→ routing / policies / entry point

Service Discovery
→ locate services

Config Server
→ centralized configuration
```

19. Why Git Backend?

A common Config Server setup stores configuration in Git:

```
Git
  ↓
version history
  ↓
Config Server
  ↓
Applications
```

Advantages:

- version control
- change history
- rollback
- review through pull requests
- auditability

Example:

```
Commit 1 → timeout=3s  
Commit 2 → timeout=5s  
Commit 3 → timeout=10s
```

You can see how configuration changed.

20. A Typical Setup

Config Server

A Spring Boot application with Config Server enabled.

Conceptually:

```
@SpringBootApplication  
@EnableConfigServer  
public class ConfigServerApplication {  
}
```

Modern Spring Cloud versions may favor auto-configuration conventions; the important thing for the interview is that this application acts as the Config Server and is connected to the configuration repository.

21. Config Client

A client service identifies itself:

```
spring:  
  application:  
    name: payment-service
```

and knows where to obtain configuration.

Modern Spring Boot/Spring Cloud commonly use Config Data import, for example conceptually:

```
spring:  
  config:  
    import: optional:configserver:http://localhost:8888
```

Older Spring Cloud setups may use:

```
spring:
```

```
cloud:
  config:
    uri: http://localhost:8888
```

This is version-dependent, so for an interview focus on the architecture rather than memorizing one version-specific property.

22. Configuration Loading Flow

At startup:

```
Payment Service starts
↓
Identify application name
↓
Identify active profile
↓
Contact Config Server
↓
Config Server reads repository
↓
Returns configuration
↓
Payment Service loads properties
↓
Application starts
```

For example:

```
payment-service + prod
↓
payment-service-prod.yml
```

23. What If Config Server Is Down?

Excellent interview question.

Suppose:

```
Payment Service
↓
Config Server ✕
```

What happens depends on how the application is configured and whether required configuration is already available locally/cached/otherwise supplied.

This is why production architectures often consider:

- startup dependencies
- local fallback configuration
- high availability
- configuration caching
- health monitoring

Don't say:

"The application always crashes."

That depends on configuration and startup behavior.

24. Why Externalize Configuration?

Bad:

```
String url =  
    "https://production-payment.example.com";
```

Better:

```
@Value("${payment.gateway-url}")  
private String gatewayUrl;
```

Now the code remains unchanged:

```
DEV → dev configuration  
UAT → UAT configuration  
PROD → prod configuration
```

This follows the principle:

Separate application code from environment-specific configuration.

25. Configuration Validation

For important settings, validate them.

Example:

```
@ConfigurationProperties(prefix = "payment")  
@Validated  
public class PaymentProperties {  
  
    @NotBlank  
    private String gatewayUrl;  
  
    @Min(100)  
    private int timeout;  
}
```

Then a bad configuration can be detected at startup rather than causing an obscure runtime failure.

This is a good production practice.

26. Important Interview Scenario

Interviewer:

"We have 50 microservices and DEV, UAT and PROD environments. How would you manage configuration?"

A strong answer:

"I'd externalize environment-specific configuration rather than hardcode it in the applications. A centralized Spring Cloud Config Server backed by a version-controlled repository can provide configuration based on application name and active profile. Services would consume their configuration at startup. Sensitive credentials should be stored in a dedicated secret-management system rather than plain Git configuration. For configuration changes that need runtime refresh, I'd use the appropriate Spring Cloud refresh mechanism, but I wouldn't assume every property can safely be changed without restarting."

27. Another Interview Question

"Why not put all configuration directly in application.yml?"

Answer:

"For a single small application, local configuration may be sufficient. In a microservice environment with multiple services and environments, centralized external configuration provides better consistency, versioning and operational management, and avoids duplicating environment-specific settings across deployments."

28. Another Interview Question

"Where should database passwords be stored?"

Don't answer:

`application.yml`

Say:

"Secrets should generally be managed through a dedicated secret-management mechanism such as Vault, AWS Secrets Manager, Azure Key Vault or Kubernetes Secrets, rather than committed as plaintext configuration."

29. Common Interview Questions

Fundamentals

1. What is configuration management?
2. Why externalize configuration?
3. What is Spring Cloud Config?
4. What is Config Server?
5. What is Config Client?
6. Why use Git as a configuration backend?

Spring

7. How does a service retrieve configuration?
8. How are profiles handled?
9. `@Value` vs `@ConfigurationProperties` ?
10. How do you validate configuration?
11. How do you refresh configuration?
12. What is `@RefreshScope` ?
13. What is Spring Cloud Bus?

Architecture

14. Config Server vs Service Discovery?
 15. Config Server vs environment variables?
 16. How do you manage secrets?
 17. What happens if Config Server goes down?
 18. How do you achieve high availability?
 19. How do you roll back a configuration change?
 20. How do you audit configuration changes?
-

30. BPS Questions — Investigate Your Codebase

Now connect the concept to your actual BPS implementation:

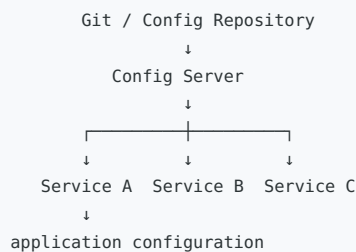
1. Where does BPS store its configuration?
2. Does BPS use Spring Cloud Config?
3. Is there a Config Server?
4. Where is the Hydra URL configured?
5. Where is the Chase URL configured?
6. How are environment-specific URLs managed?
7. Where are REST client timeouts configured?
8. Where are database connection properties configured?
9. Are secrets separated from normal configuration?

10. Does BPS use `@Value` or `@ConfigurationProperties` ?
11. Are configuration changes refreshable without restarting BPS?
12. How would you safely change a vendor endpoint in production?
13. If the Config Server is unavailable during BPS startup, what happens?
14. Which configuration belongs to BPS itself and which belongs to deployment infrastructure?

Interview-ready summary

“Spring Cloud Config provides centralized external configuration management for distributed applications. A Config Server reads configuration from a backend such as Git and exposes it to Config Clients based on application name, profile and potentially configuration label. This allows the same application code to run across different environments with different configuration. Configuration should be version controlled where appropriate, while sensitive values such as passwords and API secrets should normally be kept in a dedicated secret-management system. Spring Cloud also provides mechanisms for refreshing certain configuration at runtime. Config Server is different from service discovery: service discovery tells a service where another service is located, while Config Server tells the application what configuration it should use.”

Remember this architecture



And the key distinction:

Service Discovery → "WHERE is the service?"
Config Management → "WHAT configuration should it use?"
API Gateway → "HOW does traffic enter/routing happen?"
Secret Manager → "WHERE are sensitive secrets stored?"